Tideway: technical documentation

FinTechathon 2026 International Track, Topic E (SME Finance Copilot). Synthetic sandbox only: no real money, bank, lender, supplier or customer is connected.

1. Problem and scope

A small business with payroll approaching and cash short needs bounded, explainable choices, not another dashboard. Tideway turns business records into a source-linked cash gap, plans a recovery, negotiates with simulated counterparties and executes only what a human approved, inside a sandbox. One business uses one currency. The first version targets service businesses and wholesalers with invoices, supplier bills and payroll.

Out of scope: moving real money, production bank integration, credit decisions, payroll deferral, a full ERP, fine-tuned models.

2. Architecture

```
 Browser UI (vanilla JS)  --JSON over loopback, cookie + CSRF-->  server.py (HTTP, CSP,
rate limits, role checks)
                                                                      |
              agent.py (local planner or optional LLM; read/propose tools only)
                                                                      |
 service.py  -- analysis / propose / negotiate / what-if / ISO 20022 draft / CSV import
     |-- projection.py   exact daily cash ledger, scenarios, look-ahead
     |-- analytics.py    customer profiles, seeded Monte Carlo, KPIs, warnings, payee
checks, APR
     |-- planner.py      candidate actions, exact search, robust re-check, explanations
     |-- counterparty.py deterministic simulated suppliers and customers
     '-- store.py        SQLite: approvals, mandates, executions, audit chain, witness,
policy, sessions
 security.py (PBKDF2 PIN hashing, HMAC signing, token bucket)   domain.py (validation,
hidden-Unicode, injection scan)
```

The authority boundary is deliberate. The model and the planner live above `store.py`; only `store.py` can approve, sign, execute or change policy, and it checks the caller's role, the mandate signature, expiry and a fresh state hash on every call. Python standard library only; SQLite for storage.

3. Data model and money

Amounts are integers in minor units (`amount_minor`, two decimals). Decimal input is parsed exactly and rejected if it has more than two decimal places, is non-finite, non-positive where positive is required, or above 100,000,000. If both a decimal and a `_minor` field are supplied they must agree. Currencies are an ISO 4217 whitelist (BDT, CNY, HKD, SGD, USD, EUR, GBP, INR, MYR); a business has exactly one, and mixed currencies are rejected. Dates are real `YYYY-MM-DD` calendar dates and the horizon is at most 366 days.

Records: transactions (income/expense), invoices (draft, confirmed, overdue, paid, collected, cancelled, disputed), and bills (open, scheduled, rescheduled, paid, cancelled) with priority (payroll, critical, high, normal, low) and a `reschedule_allowed` flag. CSV import validates the whole file before writing anything (all-or-nothing), rejects unknown or duplicate columns, other tenants' rows, mixed currencies and files over 5,000 rows or 1.5 MB.

4. Projection

`projection.py` builds dated events from open invoices (inflows on expected collection dates), open bills and payroll (outflows) and accepted adjustments, then walks a daily ledger: closing cash is the previous day's cash plus inflows minus outflows. Settled invoices are never counted again. Three scenarios share the engine:

- **Scheduled** (base): the dates in the records.
- **Likely**: each open invoice moves to the customer's expected date from their payment profile.
- **Stress**: a pessimistic (high-percentile) delay per customer.

The planning horizon is the target date plus a look-ahead (default 14 days, 0 to 60), so a plan that only pushes the shortfall past the target date is not counted as a fix. Shapla Services (fixture): opening cash BDT 120,000, payroll 80,000, supplier bill 70,000, confirmed receivable 20,000, reserve floor 20,000. Scheduled minimum cash is **BDT -10,000 on 31 Oct 2026**; the reserve shortfall is **BDT 30,000**.

5. Customer payment profiles

From settled invoices, lateness is `paid_date − due_date`. For each customer the expected delay is a Buhlmann-style credibility blend: own history carries weight n/(n+k) with k = 5, and the business-wide mean carries the rest. The profile reports grade (A to D), confidence (low under 3 settled invoices, medium under 8, high otherwise), on-time rate, p80/p90 delay and a plain-language explanation. Customers with no history fall back to the portfolio, and a business with no history at all to a stated default. A back-test on settled invoices is shown beside the forecast; no accuracy claim is made beyond that synthetic data.

6. Monte Carlo risk

Only payment timing is uncertain. For each of 1,000 runs, one delay is drawn per customer (a slow customer is slow on all their invoices) from that customer's history mixed with the portfolio; no invoice is paid before its expected date; everything else is fixed. The output is the probability of breaching the reserve floor, of negative cash and of unfunded payroll, expected and p90 shortfall, minimum-cash percentiles, the most common breach date and a fan of daily percentiles. The seed is a hash of business, snapshot hash and target date, and the code uses only `Random.random()` (the stream Python guarantees), so the same records always give the same numbers and tests assert exact results. The UI summarises the result as cash weather: Clear, Unsettled (breach probability of at least 15%) or Storm (a scheduled breach, or at least 50%).

7. Planner (exact, risk-aware)

Candidate actions per record, all within policy:

- **Supplier deferral**: move 50% or 100% of a bill 7 or 14 days later, only if `reschedule_allowed`, never payroll, never a blocked payee. Priced at an assumed 12% a year.
- **Early-payment offer**: offer a customer a 1, 2 or 3% discount to pay earlier; priced as an annualised rate, `d/(1−d) x 365/days` (2% for 20 days is about 3,724 bps).
- **Collection reminder**: a zero-cash draft; a promise to pay is tracked as information, never as cash.
- **Financing**: three labelled MOCK products (invoice advance, working-capital line, guaranteed loan) shown with fee, term and time to cash. Never executable.

Search: every combination of at most one option per record, over the six most relevant records, is evaluated with a fast daily ledger. A bundle is *feasible* if the reserve floor holds on every day of the horizon and payroll is funded; bundles above the daily cap are dropped. Feasible bundles are ranked by shortfall, then cost, then number of counterparties, and the best 40 are re-checked with a seeded 400-run Monte Carlo. The recommendation is the cheapest bundle whose simulated breach probability is within the 10% risk target; otherwise the safest (risk compared in 5-point buckets to absorb sampling noise). Alternatives are chosen to differ in counterparties. If no feasible bundle exists, the explanation says so and escalates. The planner returns structured reasons for every rejected record (for example `SECURITY_BLOCKED`, `DECLINED`).

8. Negotiation with simulated counterparties

`counterparty.py` plays suppliers (styles flexible, standard, cautious, strict) and customers with fixed, repeatable rules and replies **accepted**, **countered** (with terms inside the counterparty's own limits) or **rejected**; a reminder gets an acknowledgement with a promise date. Each reply is labelled simulated. A counter is turned into a revised proposal that needs fresh approval. A rejection excludes that counterparty and triggers a re-plan. Execution requires every action in the mandate to have an accepted reply first.

9. Mandates, approval and execution

1. **Propose**: the agent creates a plan with up to three bundles (recommended and alternatives), each with exposure, risk before and after, and the source records.
2. **Approve** (`store.approve`): the operator re-enters their PIN (PBKDF2, 120,000 iterations; five failures lock the operator for five minutes). Viewers cannot approve. If exposure exceeds the dual-approval threshold, two different people must approve (one of them the owner), and each must type the exact total exposure to show they read it. Approvals made in under two seconds are flagged as possible rubber-stamping in the audit record. The second approval issues the mandate.
3. **Mandate**: a JSON document signed with HMAC-SHA256, modelled on the AP2 mandate idea (cryptographically verifiable, scoped to specific actions, short-lived). It binds actions, amounts, counterparties, plan and snapshot hashes, policy version and a 15-minute expiry. It can be verified offline by pasting the document.
4. **Execute** (`store.execute`): checks the signature, expiry, that the caller is a human approver (or the agent under a standing mandate, for zero-cash reminders only), that the agent is not paused, that all counterparties accepted, and that the underlying records and state hash have not changed since approval. Execution runs in one `BEGIN IMMEDIATE` transaction, stores a receipt with before/after state hashes, then reads the state back. The same idempotency key returns the same receipt; a new key on a consumed mandate is refused.

10. Controls

- **Kill switch / circuit breaker**: pausing blocks execution and planning; only the owner resumes, with a stated reason. Five security denials within ten minutes pause the agent automatically.
- **Payee fraud**: changed bank details and look-alike names (name similarity of 0.85 or more to a different known payee) block all actions on that bill. Verification is owner-only, by callback to a number on file, with a note; the verifier cannot approve that payee.
- **Injection and hidden characters**: text is NFKC-normalised, invisible format characters (category Cf) are stripped and flagged; patterns such as "ignore previous instructions", "pay now to this account", "bypass approval", new bank details, markup and links are quarantined. Quarantined records are excluded from planning and never sent to a model.
- **Autonomy dial**: Advise (read and explain), Propose (create plans), Delegate (a standing mandate, seven days, lets the agent send zero-cash reminders, five a day). The matrix is shown in the Security Centre. Moving money and payroll deferral are never delegated.
- **Policy**: reserve floor, per-action limit, daily cap, dual-approval threshold, look-ahead, pause flag. Changes need the PIN.

11. Agent and optional LLM mode

Six tools: `read_ledger`, `profile_customers`, `project_cash`, `simulate_risk`, `check_security`, `compare_plans` (creates a proposal; never approves). Maximum eight tool calls per request. The default **local agent** is a deterministic planner that runs a visible step-by-step rescue run and answers questions from tool output, with no network. The optional **LLM mode** (OpenAI Responses API, user key, per-message consent) sends only pseudonymised counterparty names and tool output, never record free text; tool results are framed as data; every number in the answer is compared against tool output and ungrounded figures are flagged. Hijacked-model behaviour is tested with a simulated transport (see the security self-assessment). No live-model measurement has been made.

12. Audit and evidence

Every proposal, approval, negotiation, execution and rejection is appended to a per-business hash chain (each entry hashes its predecessor). An independent **witness** file with its own key periodically co-signs the chain head, so an attacker who rewrites the database to hide entries, including deleting the tail, is detected. The Evidence view verifies the chain, shows the first invalid entry and offers a **tamper drill** that edits a copy and shows detection. Exports: evidence JSON, records CSV and receipts CSV (cells beginning `=`, `+`, `-` or `@` are prefixed with `'`), and a draft **ISO 20022 pain.001.001.09** file for deferred supplier payments. The draft is never transmitted.

13. HTTP layer

Loopback only. Host and Origin allow-lists, a per-run CSRF token on every write, `HttpOnly; SameSite=Strict` session cookie (eight hours), JSON-only bodies up to 2 MB, strict field allow-lists per endpoint, role checks, token-bucket rate limits (login stricter), no-store caching and a strict Content-Security-Policy, `X-Frame-Options: DENY`, `nosniff`. Internal errors are logged to the console and not returned.

14. UI

A single-page app with seven views: Overview (cash weather, scenarios, fan chart, warnings), Rescue (agent run, plan bundles, approval, negotiation, receipt), Assistant, What-if (run changes without writing), Records (import, quarantine), Security (permission tiers, autonomy dial, kill switch, payee verification, self-test) and Evidence (audit chain, exports, tamper drill).

15. Limitations (honest)

Synthetic data; counterparties are rules, not people; the Monte Carlo models payment timing only; the back-test is on synthetic history; PINs and the signing key are demo-grade (a production system needs real identity, hardware-backed keys and a bank rail); the hash chain and witness reduce, but do not eliminate, risk from a fully privileged attacker; no live-LLM or user-study results are claimed; this is not a compliance certification.